# Lab Report No 9
# Implementation of Color Image Processing – II
## Digital Image Processing


**Submitted By:**

[Huzaifa Shahbaz]

[21-CE-009]

[Aqsa Shah]

[21-CE-039]

**Submitted to:**

Engr. Hareem Khan

**Dated:**

6th Dec,2024

**Department of Computer Engineering,**

**HITEC University, Taxila**

## Lab Example No 1:

**Solution:**

| Brief description (3-5 lines) |
| --- |
| **RGB SLICING:**<br>RGB slicing technique used in image processing which help us as one to remove the specific color components such as Red, Green, or Blue from mages. Pixels are represented with RGB values which define their color intensity for the three primary color channels. When the two channels are set to zero while the other channel's intensity is maintained, a slice is obtained. This process would help the analysis of specific features within an image concerning the color in it. RGB slicing is widely used by a number of applications such as object detection, segmentation, or enhancing visual data. |
| **The code** |
| import cv2<br>import numpy as np<br>import matplotlib.pyplot as plt<br># Read the RGB image<br>image = cv2.imread('RGB Slicing.jpg')<br>image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB) # Convert BGR to RGB<br>plt.figure()<br>plt.imshow(image)<br>plt.title("Original Image")<br>plt.axis('off')<br>w = 140<br>a = np.array([30, 30, 30])<br># Create a copy of the image for modification<br>sliced_image = image.copy()<br># Vectorized approach for RGB slicing<br>diff = np.abs(image - a) # Calculate absolute differences for all pixels<br>mask = np.any(diff > w / 2, axis=-1) # Check if any channel exceeds w/2 |

```
sliced_image[mask] = 128 # Set gray value for pixels that match the condition
# Display the result
plt.figure()
plt.imshow(sliced_image)
plt.title("RGB Sliced Image (Optimized)")
plt.axis('off')
plt.show()
```

**The results (Screenshot)**

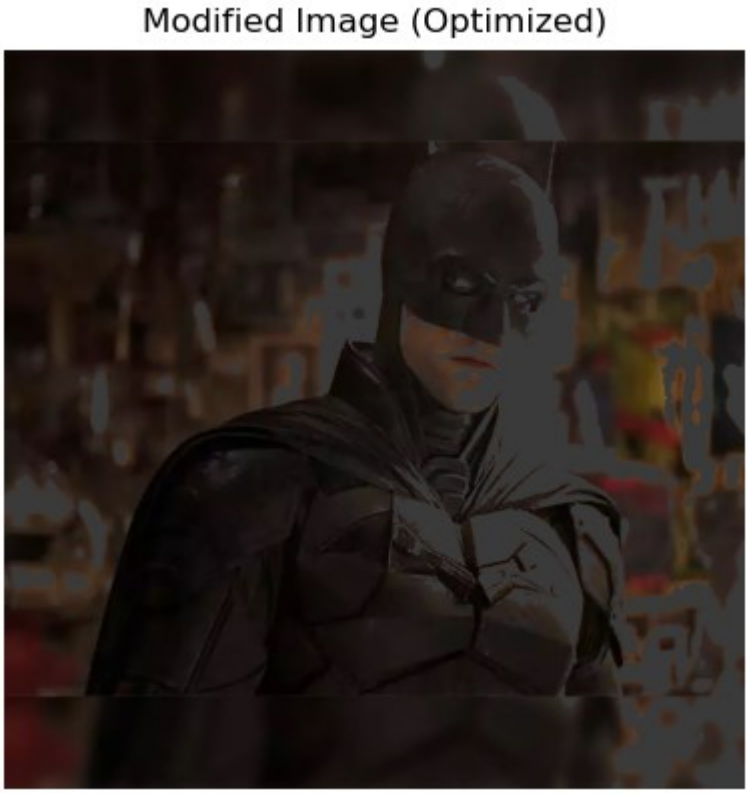

## Lab Example No 2:

**Solution:**

| Brief description (3-5 lines) |
| --- |
| **RGB SLICING:**<br>RGB slicing technique used in image processing which help us as one to remove the specific color components such as Red, Green, or Blue from mages. Pixels are represented with RGB values which define their color intensity for the three primary color channels. When the two channels are set to zero while the other channel's intensity is maintained, a slice is obtained. This process would help the analysis of specific features within an image concerning the color in it. RGB slicing is widely used by a number of applications such as object detection, segmentation, or enhancing visual data. |
| **The code** |
| import cv2 |

```
import numpy as np
import matplotlib.pyplot as plt
# Read the RGB image
image = cv2.imread('RGB Slicing.jpg')
image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB) # Convert
BGR to RGB for correct visualization
# Display the original image
plt.figure()
plt.imshow(image)
plt.title("Original Image")
plt.axis('off')
# Create a copy of the image for modification
modified_image = image.copy()
# Vectorized approach
mask = (image[:, :, 1] > 40) & (image[:, :, 2] > 40) # Condition for
green and blue channels
modified_image[mask] = 58 # Apply the gray value to pixels that
satisfy the condition
# Display the modified image
plt.figure()
plt.imshow(modified_image)
plt.title("Modified Image (Optimized)")
plt.axis('off')
plt.show()
```

**The results (Screenshot)**




## Lab Example No 3(a):

**Solution:**

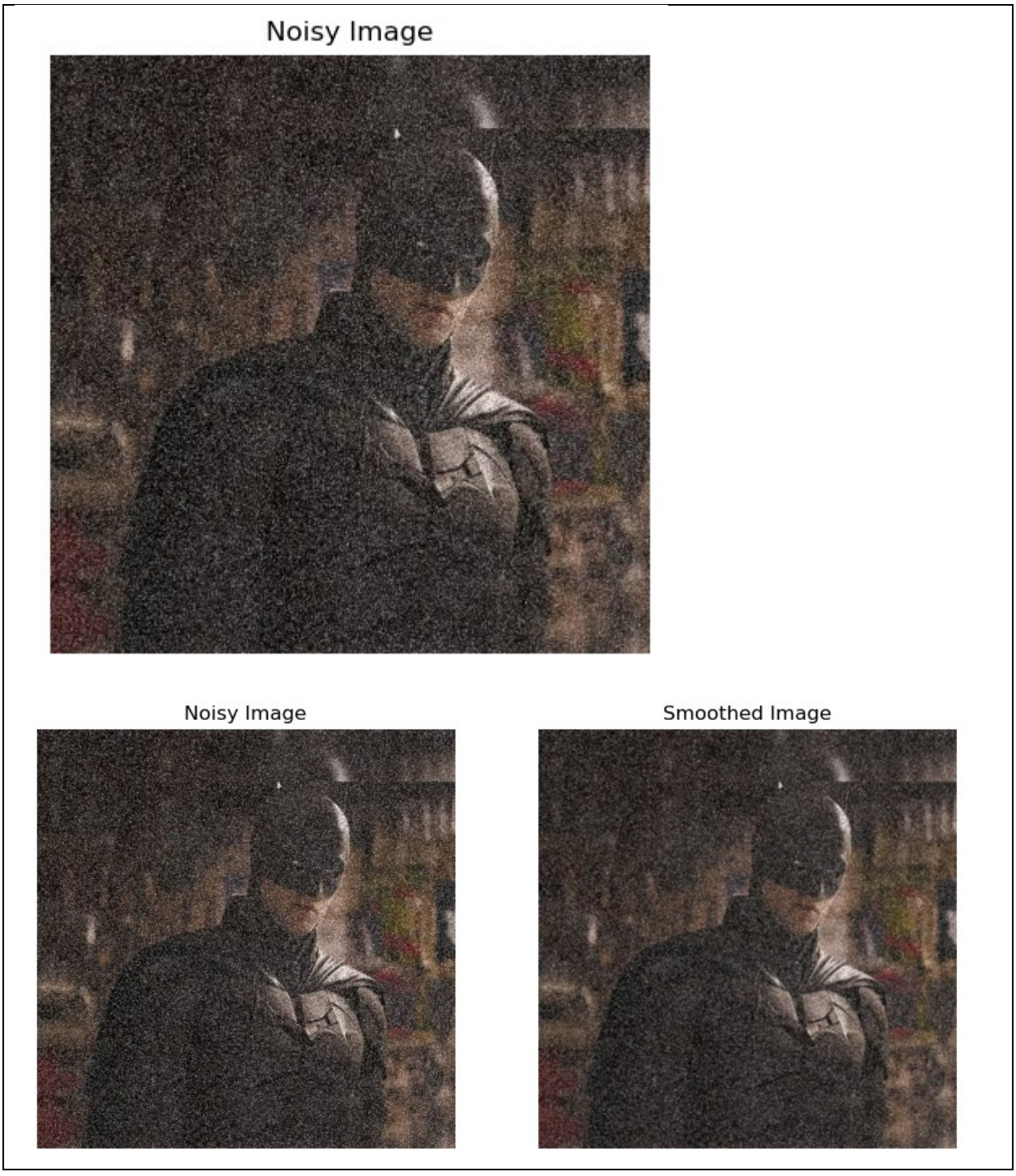
| Brief description (3-5 lines) |
| --- |
| **RGB SMOOTHING:**<br>RGB smoothing is an image processing technique for eliminating noise and sharp transitions from within an image by averaging the neighboring color values of a pixel with its neighbors. Filters such as Gaussian or mean filters are applied either separately or in combination across the three RGB channels. Smoothing of the Red, Green, and Blue channels thus renders the image less pixelated and more visually coherent, while the overall color composition stays intact. Applications for RGB smoothing are many; they include image enhancement, pre-processing for computer vision tasks, and enhancement of image quality for photography and video. |
| **The code** |
| import cv2<br>import numpy as np<br>import matplotlib.pyplot as plt |

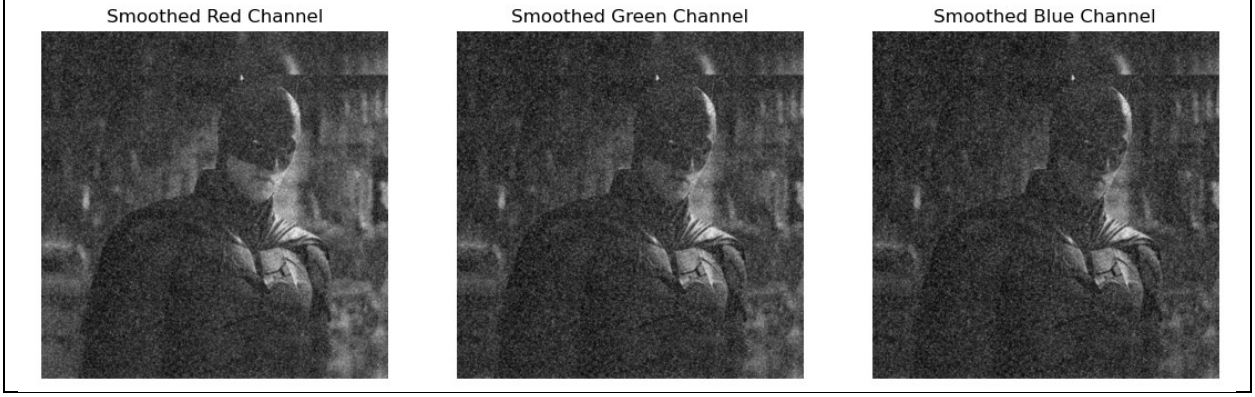
```
# Step 1: Load the RGB image
image = cv2.imread('RGB SMOOTHING.jpg') # Read the image
image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB) # Convert
fromBGR (OpenCV format) to RGB
plt.figure()
plt.imshow(image)
plt.title("Original Image")
plt.axis('off')
# Step 2: Add salt-and-pepper noise
def add_salt_pepper_noise(image, noise_level):
    noisy_image = image.copy() # Make a copy of the image to modify
    num_pixels = int(noise_level * image.size * 0.5) # Half for salt, half
for pepper
    # Add salt (white pixels)
    salt_coords = [np.random.randint(0, dim, num_pixels) for dim in
                image.shape[:2]]
    noisy_image[salt_coords[0], salt_coords[1], :] = 255 # Set to white
    # Add pepper (black pixels)
    pepper_coords = [np.random.randint(0, dim, num_pixels) for dim in
                  image.shape[:2]]
    noisy_image[pepper_coords[0], pepper_coords[1], :] = 0 # Set to
black
    return noisy_image
# Apply salt-and-pepper noise with 12% corruption
noisy_image = add_salt_pepper_noise(image, 0.12)
# Display the noisy image
plt.figure()
plt.imshow(noisy_image)
plt.title("Noisy Image")
plt.axis('off')
# Step 3: Create the average filter
# The filter is a 5x5 matrix where all values are 1/25
avg_filter = np.ones((5, 5)) / 25.0
# Step 4: Apply the filter to each color channel
# Separate the Red, Green, and Blue channels
```

```
R = noisy_image[:, :, 0]
G = noisy_image[:, :, 1]
B = noisy_image[:, :, 2]
# Use OpenCV's filter2D to apply the average filter
R_smooth = cv2.filter2D(R, -1, avg_filter) # Smooth the Red channel
G_smooth = cv2.filter2D(G, -1, avg_filter) # Smooth the Green
channel
B_smooth = cv2.filter2D(B, -1, avg_filter) # Smooth the Blue channel
# Step 5: Combine the smoothed channels back into one image
smoothed_image = cv2.merge([R_smooth, G_smooth, B_smooth])
# Step 6: Display the results
# Display the noisy and smoothed images side by side
plt.figure(figsize=(10, 5))
# Show noisy image
plt.subplot(1, 2, 1)
plt.imshow(noisy_image)
plt.title("Noisy Image")
plt.axis('off')
# Show smoothed image
plt.subplot(1, 2, 2)
plt.imshow(smoothed_image.astype(np.uint8))
plt.title("Smoothed Image")
plt.axis('off')
plt.show()
# Step 7: Visualize each channel
plt.figure(figsize=(15, 5))
# Red channel
plt.subplot(1, 3, 1)
plt.imshow(R_smooth, cmap='gray')
plt.title("Smoothed Red Channel")
plt.axis('off')
# Green channel
plt.subplot(1, 3, 2)
plt.imshow(G_smooth, cmap='gray')
plt.title("Smoothed Green Channel")
```

```
plt.axis('off')
# Blue channel
plt.subplot(1, 3, 3)
plt.imshow(B_smooth, cmap='gray')
plt.title("Smoothed Blue Channel")
plt.axis('off')
plt.show()
```

**The results (Screenshot)**


Noisy Image
Noisy Image
Smoothed Image




## Lab Example No 4:

**Solution:**

| Brief description (3-5 lines) |
| --- |
| **Intensity Slicing:**<br>Intensity slicing is an image processing technique that highlights certain pixel intensity ranges in a grayscale image. A portion of the intensity range can be associated with either color or brightness, creating pixel intensity-based segmentation of the image. This method is very useful in the consideration of various features of interest, as it enables the specification of colors for only those pixels that fall within the range of the feature of interest in the intensity and leaves the rest unchanged. Intensity slice is a common technique used to enhance the visibility of certain structures or patterns in the obtained images, especially in medical imaging, remote sensing, and image segmentation. |
| **The code** |
| import numpy as np<br>import cv2<br>import matplotlib.pyplot as plt<br># Step 1: Load the grayscale image<br>image = cv2.imread('Intensity Slicing.jpg',cv2.IMREAD_GRAYSCALE) # Load as grayscale<br>plt.figure()<br>plt.imshow(image, cmap='gray')<br>plt.title("Original Image") |

```
plt.axis('off')
# Step 2: Create an RGB image initialized with zeros
I_rgb = np.zeros((image.shape[0], image.shape[1], 3), dtype=np.uint8)
# Step 3: Perform intensity slicing
for i in range(image.shape[0]):
    for j in range(image.shape[1]):
        intensity = image[i, j]
        if 0 <= intensity < 32:
            I_rgb[i, j, 0] = 64 # Red channel
            I_rgb[i, j, 2] = 64 # Blue channel
        elif 32 <= intensity < 64:
            I_rgb[i, j, 1] = 64 # Green channel
            I_rgb[i, j, 2] = 64 # Blue channel
        elif 64 <= intensity < 96:
            I_rgb[i, j, 0] = 96 # Red channel
            I_rgb[i, j, 2] = 96 # Blue channel
        elif 96 <= intensity < 128:
            I_rgb[i, j, 1] = 96 # Green channel
            I_rgb[i, j, 2] = 96 # Blue channel
        elif 128 <= intensity < 160:
            I_rgb[i, j, 0] = 128 # Red channel
            I_rgb[i, j, 2] = 128 # Blue channel
        elif 160 <= intensity < 192:
            I_rgb[i, j, 0] = 255 # Red channel
        elif 192 <= intensity < 224:
            I_rgb[i, j, 1] = 255 # Green channel
        elif 224 <= intensity < 256:
            I_rgb[i, j, 2] = 255 # Blue channel
# Step 4: Display the color-mapped image
plt.figure()
plt.imshow(I_rgb)
plt.title("Intensity Sliced Image")
plt.axis('off')
plt.show()
```

**The results (Screenshot)**

Original Image

Intensity Sliced Image

**Conclusion**

In conclusion, manipulation and analysis of color images using advanced techniques can be achieved through Color Image Processing - II. Enhancement, segmentation, and transformation are specially expanding components of color image processing. The need for color models, including RGB and HSV, will be ushered in when image data processing reflects without misrepresentation to real-world applications. The results indicate better quality images and feature extraction, which can therefore be very useful in computer vision and multimedia. On this project, the efficient algorithm handling large data sets and performing brilliantly is aimed at. It is basically setting a pace for future research on more complex images being processed.